AI Driven

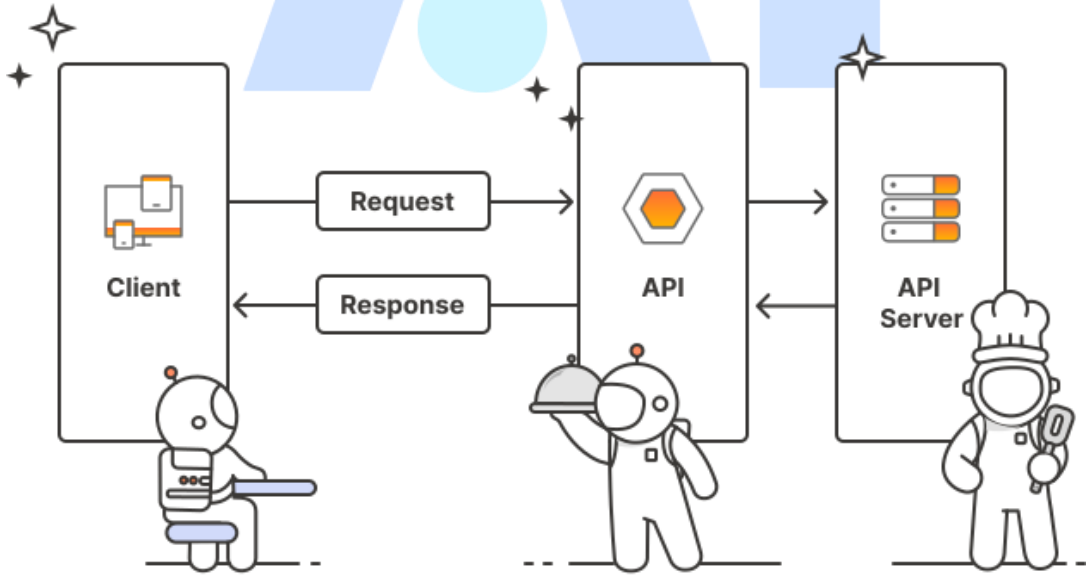
Application Programming Interface

An API, which stands for **Application Programming Interface**, is a set of protocols that enable **different software components to communicate** and **transfer data**. Developers use APIs to bridge the gaps between small, discrete chunks of code in order to create applications that are powerful, resilient, secure, and able to meet user needs. Even though you can't see them, **APIs are everywhere-working continuously** in the **background to power** the **digital experiences** that are essential to our modern lives.

Simple meaning

API is a communication bridge between two software systems.

One system **sends a request** → another system **sends a response**.



Swiggy la "**Order Food**" press panna → App → Server → Food data return → App show pannu.

Ithu API work.

AI Driven

1. Why API is used?

- To connect frontend & backend
- To send/receive data between apps
- To login/signup
- To display products, users, orders
- For automation testing
- For mobile app communication
- For microservices communication

2. Where API is used?

Everywhere:

- Websites
- Mobile apps
- Banking apps
- E-commerce
- Payment gateways
- Social media
- Gaming apps
- IoT devices

3. Types of APIs

Based on Access

1. Public API (*Open API*)

Definition:

APIs that are **publicly available** for **anyone to use**.

Features:

- Free or paid
- No special permission required
- Mostly used for public data

Example (Real Life):

- OpenWeather API – shows weather details to any app.

Example (Endpoint):

https://api.openweathermap.org/data/2.5/weather?q=Chennai

2. Private API

Definition:

APIs used only within a **specific company** or **organization**.

Features:

- Not accessible to public
- High security
- Used for internal systems

Example (Real Life):

- Company HRMS API for attendance, payroll, employee details.

Example (Endpoint):

https://internal.company.com/hr/employees

3. Partner API

Definition:

APIs shared only with **business partners** or **clients.**

Features:

- Access given through API keys
- Used for business-to-business integration
- Limited access

Example (Real Life):

- Paytm Merchant API (for businesses only)

Example (Endpoint):

https://api.paytm.com/merchant/v3/payment

AI Driven

Based on Architecture

1. REST API (Most Common)

Definition:

- A simple, lightweight API architecture using HTTP methods.
- Mostly returns JSON format.

Features:

- Easy to use
- Fast
- Works with GET, POST, PUT, DELETE

Example (Real Life):

- Swiggy product list
- Instagram posts
- YouTube videos list

Example (Endpoint):

GET https://api.example.com/users

JSON Response:

```
{
  "id": 1,
  "name": "Giri"

}
```

2. SOAP API

Definition:

- A protocol-based API using XML format.
- Older and more secure.

Features:

- Uses XML only
- High security
- Heavy and slow compared to REST

Example (Real Life):

- Banking services
- Insurance systems

SOAP Request Example:

```
<soap:Envelope>
  <soap:Body>
    <getUser>
      <id>101</id>
    </getUser>
  </soap:Body>
</soap:Envelope>
```

3. GraphQL API

Definition:

A query-based API where the client requests only the required data.

Features:

- Single endpoint
- No over-fetching
- No under-fetching

Example (Real Life):

- Facebook
- GitHub
- Shopify

GraphQL Query Example:

```
{
  user(id: 1) {
    name
    email
    posts {
      title
    }
  }
}
```

4. gRPC API

Definition:

A high-performance API framework using binary protocol (Protocol Buffers).

Features:

- Very fast
- Used in microservices
- Works on HTTP/2

Example (Real Life):

- Netflix internal services
- Google systems
- Cloud microservices

Protobuf Example:

```
message User {
  int32 id = 1;
  string name = 2;
}
```

4. Quick Comparison

Type	Data Format	Speed	Usage
REST	JSON	Medium	Web & mobile apps
SOAP	XML	Slow	Bnotranking, enterprise
GraphQL	JSON	Medium	Modern apps
gRPC	Binary	Very Fast	Microservices

5. REST API Basic Concepts

HTTP Methods:

Method	Meaning	Usage
GET	Fetch data	Get user list
POST	Create data	Add user
PUT	Update entire data	Update full user
PATCH	Update partial data	Change only name
DELETE	Remove data	Delete user

6. API Request Structure

API request **Three main** things need:

1. URL (Endpoint)
2. Method
3. Body/Parameters/ Header

AI Driven

1. URL (*Endpoint*)

Example:

```
https://api.example.com/users
```

2. Method

GET /POST /PUT /DELETE

3. Body / Parameters / Headers

- Body → Data (mostly JSON)
- Params → Filters
- Headers → Auth, Content-Type

```
{
  "name": Giri,
  "name": "mrgiri@gamil.com"
}
```

7. API Response Structure

Most common response = JSON

```
{
  "status": "success",
  "data":
        {
              "id": 101,
              "name": "Giri"
        }
}
```